

HELIXDEVELOPMENT

HelixAgent Comprehensive Implementation & Verification Plan

Universal Strategy for 100% Functional Success Across All Submodules

Project: github.com/HelixDevelopment/HelixAgent

Scope: 58 Submodules | 60+ Internal Packages | 47+ LLM Providers

Date: April 2026

Classification: Engineering Master Plan

Table of Contents

1. Executive Summary

2. Project Architecture Overview

2.1 System Topology

2.2 Technology Stack

2.3 Data Flow Architecture

3. Submodule Registry (All 58 Submodules)

3.1 Core Foundation Layer

3.2 Memory & Context Layer

3.3 LLM & Provider Layer

3.4 Verification & Quality Layer

3.5 Tooling & Integration Layer

3.6 Security & Governance Layer

3.7 Deployment & Operations Layer

3.8 Interface & Experience Layer

4. Internal Package Inventory

5. The Universal Verification Framework

5.1 Institutional Knowledge Mapping

5.2 Testing Pyramid Rebalance

5.3 Verification Loop Protocol

5.4 LLM-as-Judge Layer

5.5 CLAUDE.md Operating System

5.6 Multi-Agent Review Process

6. Phase-by-Phase Implementation Plan

Phase 0: Foundation & Audit

Phase 1: Verification Infrastructure

Phase 2: Core Submodule Hardening

Phase 3: Memory & Context Systems

Phase 4: LLM Provider Integration

Phase 5: Tooling & MCP Ecosystem

Phase 6: Verification & QA Systems

Phase 7: Security & Governance

Phase 8: Deployment & Operations

Phase 9: Interface & SDK Hardening

Phase 10: Continuous Validation

7. Submodule-Specific Action Items

8. Testing Strategy for 100% Functional Correctness

9. Operational Runbooks

10. Risk Register & Mitigation

11. Success Metrics & Exit Criteria

12. Appendices

1. Executive Summary

HelixAgent is a production-grade, AI-powered ensemble LLM service written in Go that aggregates responses from 47+ language model providers to produce optimal, verified outputs. The project comprises **58 Git submodules**, **60+ internal Go packages**, multi-platform deployment targets (Linux, macOS, Windows, Android, Web), and a sophisticated memory/context architecture.

CRITICAL PROBLEM STATEMENT

Despite near-100% unit test coverage, manual testing reveals widespread functional failures. Tests pass in isolation, but integrated behavior is broken. This document provides a **universal, mechanical, repeatable system** to achieve 100% functional correctness across every submodule and internal package.

Key Statistics

Metric	Value	Target State
Git Submodules	58	All functional, synced, documented
Internal Packages	60+	All passing integration tests
LLM Providers	47+	All verified with live API calls
MCP Implementations	35	All interoperable
Lines of Go Code	~500K+	100% lint-free, zero gosec issues
Test Files	1000+	Unit + Integration + E2E + Property
Container Services	24+	All health-checked, orchestrated

The Universal Solution: Replace Trust with Verification

The solution is not to abandon AI-assisted development, but to build a **multi-layered verification fortress** around it. The approach consists of eight interconnected layers:

1. **Institutional Knowledge Codification** — Turn implicit rules into explicit, machine-readable constraints.
2. **Testing Pyramid Rebalance** — Move beyond unit tests to contract, behavioral, property-based, and chaos tests.
3. **Self-Correcting Verification Loop** — Force AI agents to run full test suites and fix their own failures.
4. **LLM-as-Judge Gatekeeper** — A second AI instance reviews every change before merge.
5. **CLAUDE.md Operating System** — Persistent project context that every agent must read before acting.
6. **Multi-Agent Review Process** — Separate author, strategy, and structure reviewer roles.
7. **Operational Guardrails** — Feature flags, environment-specific testing, and progressive rollout.
8. **Continuous Learning Loop** — Every bug found in manual testing becomes a new automated test or landmine rule.

2. Project Architecture Overview

2.1 System Topology

HelixAgent follows a **layered microservices architecture** with clear separation of concerns:

Layer	Purpose	Key Components
Entry Layer	API surface, CLI, WebSocket	cmd/, pkg/api, extensions/vscode
Transport Layer	HTTP/gRPC/WebSocket handling	internal/handlers, internal/http, internal/transport
Business Logic Layer	Core orchestration, ensemble logic	internal/ensemble, internal/planning, internal/llm,
Provider Layer	LLM provider adapters	internal/llm/providers/, HelixLLM, LLMProvider
Memory Layer	Short-term and long-term memory	HelixMemory, Memory, internal/memory, VectorDB
Tool Layer	MCP servers, skills, plugins	MCP, ToolSchema, SkillRegistry, internal/tools
Verification Layer	Quality assurance, testing	LLMsVerifier, HelixQA, Veritas
Security Layer	Auth, encryption, audit	Auth, Security, RedTeam, internal/security
Infrastructure Layer	Storage, cache, messaging	Database, Cache, Storage, Messaging, EventBus
Observability Layer	Metrics, logging, tracing	Observability, Benchmark, monitoring/

2.2 Technology Stack

Category	Technology	Version/Notes
Primary Language	Go	1.24+ (per go.mod), targeting 1.26
Build System	Make + Custom Scripts	1976-line Makefile
Container Runtime	Docker / Podman	24+ compose files for different profiles
Orchestration	Kubernetes	Helm charts in k8s/ and deployments/
Database	PostgreSQL	Primary persistent store
Cache	Redis	L1/L2 tiered cache
Message Queue	Kafka	Event sourcing, streaming
Vector Store	Multiple (Qdrant, Weaviate, pgvector)	Via VectorDB submodule
Web Frontend	React + TypeScript	Website submodule
Mobile	Android (Kotlin)	sdk/ for Android, iOS
Security Scanning	gosec, trivy, snyk, semgrep	Integrated in Makefile
Testing	go test, testify, gopter	Property-based testing with gopter

2.3 Data Flow Architecture

CRITICAL PATH (Must Never Break)

User Request -> API Gateway (Envoy) -> Go Backend -> Request Router -> Ensemble Engine -> Provider Adapters -> External LLM APIs -> Response Aggregation -> Memory Store -> VectorDB -> Return to User

The ensemble engine is the heart of HelixAgent. It:

1. Receives a user query via HTTP/gRPC/WebSocket
2. Retrieves relevant context from VectorDB and Memory

3. Distributes the query to multiple LLM providers concurrently
4. Collects and scores responses using the Veritas scoring engine
5. Optionally runs debate orchestration (DebateOrchestrator) for contentious queries
6. Applies post-processing (formatting, normalization, fact-checking)
7. Stores the interaction in Memory for future context
8. Returns the optimized response to the user

3. Submodule Registry (All 58 Submodules)

Every submodule is listed below with its purpose, criticality, upstream URL, and specific action items required for 100% functional correctness. **No submodule is omitted.**

3.1 Core Foundation Layer

3. Submodule Registry (All 58 Submodules)

Submodule	Purpose	Criticality	Upstream	Action Items
Containers	Docker/container orchestration utilities, lazy orchestrator patterns	P0	vasic-digital/Containers	Verify LazyOrchestrator thread-safety (safe.Store migration CONST-029). Validate all container lifecycle hooks. Integration test with actual Docker daemon.
Concurrency	Concurrent-safe primitives, deadlock detector, worker pools	P0	vasic-digital/Concurrency	Verify deadlock detection accuracy under load. Test WorkerPool with 10K+ goroutines. Validate safe.Store atomic operations.
Database	PostgreSQL abstraction, query builder, migration system	P0	vasic-digital/Database	Verify connection pooling under stress. Test QueryCache safe.Store migration. Validate migration rollback paths.
Storage	Object storage abstraction (S3, GCS, Azure Blob)	P0	vasic-digital/Storage	Test upload/download with 1GB+ files. Verify multipart upload resume. Test all three provider backends.

3. Submodule Registry (All 58 Submodules)

Cache	Tiered caching (L1 in-memory, L2 Redis)		P0	vasic-digital/Cache	Verify L1/L2 coherence. Test cache stampede protection. Validate tag-based invalidation with safe.Store.
Messaging	Kafka producer/consumer, DLQ, replay system		P0	vasic-digital/Messaging	Test DLQ processing under failure. Verify exactly-once semantics. Validate message replay from checkpoints.
EventBus	Internal pub/sub event system		P1	vasic-digital/EventBus	Verify event ordering guarantees. Test subscriber cleanup (no goroutine leaks). Validate backpressure handling.
PliniusCommon	Shared common constants	utilities, types,	P0	vasic-digital/PliniusCommon	Verify all shared types are serialization-safe. Test cross-submodule compatibility after updates.

3.2 Memory & Context Layer

3. Submodule Registry (All 58 Submodules)

Submodule	Purpose	Criticality	Upstream	Action Items
HelixMemory	Primary memory system: short-term, long-term, episodic, semantic	P0	vasic-digital/HelixMemory	Verify fusion duration and confidence merging (recent fix). Test memory retrieval accuracy. Validate distributed memory with Kafka replay.
Memory	Memory subsystem interface and adapters	P0	vasic-digital/Memory	Test Mem0 adapter integration. Verify memory adapter test suite (fix infra compose). Validate Cogneer mock server migration to safe.Store.
VectorDB	Vector database abstraction (Qdrant, Weaviate, Chroma, pgvector)	P0	vasic-digital/VectorDB	Test vector similarity search with 1M+ vectors. Verify all provider backends. Test hybrid search (sparse + dense).

3. Submodule Registry (All 58 Submodules)

Embeddings	Embedding generation and management (13 providers)	P0	basic-digital/Embeddings	Verify consistency across providers. Test embedding with 10K documents. Validate dimension normalization.
ConversationContext	Multi-turn conversation state management	P1	basic-digital/conversation	Test context window management (sliding, summarization). Verify context serialization/deserialization. Test multi-user isolation.
RAG	Retrieval-Augmented Generation pipeline	P0	basic-digital/RAG	Verify synonym handling (safe.Store migration). Test end-to-end retrieval accuracy. Validate reranking pipeline.

3.3 LLM & Provider Layer

3. Submodule Registry (All 58 Submodules)

Submodule	Purpose	Criticality	Upstream	Action Items
HelixLLM	Primary LLM abstraction and provider management	P0	vasic-digital/HelixLLM	Verify generic 3-tier model discovery (recent fix). Refactor all 10 stale provider configurations. Test streaming vs non-streaming paths.
LLMProvider	Provider-agnostic LLM interface	P0	vasic-digital/LLMProvider	Test all provider adapter initialization paths. Verify fallback chaining. Validate rate limit handling across all providers.
LLMOrchestrator	Multi-LLM request distribution and load balancing	P0	vasic-digital/LLMOrchestrator	Test circuit breaker behavior. Verify request routing to model capability. Test failover under provider outages simulation.
LLMOps	Operational tooling for LLM deployments	P1	vasic-digital/LLMOps	Verify InMemoryContinuousEvaluation safe.Store migration. Implement prompt version management. Validate A/B test framework.
I-LLM	Inference optimization and caching for LLMs	P1	vasic-digital/I-LLM	Verify GPTCache semantic caching (Pattern-matching migration). Test inference result caching. Validate cache invalidation on model updates.
Models	Model definitions, schema validation	P0	vasic-digital/Models	Verify all model schemas against provider API docs. Implement model capability detection. Validate model metadata freshness.
HyperTune	Hyperparameter tuning for model selection	P2	vasic-digital/HyperTune	Test automated model selection accuracy. Verify tuning convergence. Validate against benchmark datasets.

3. Submodule Registry (All 58 Submodules)

AutoTemp	Dynamic temperature and sampling adjustment	P2	vasic-digital/AutoTemp	Verify temperature adaptation logic. Test creative vs deterministic task routing. Validate sampling parameter bounds.
-----------------	---	----	------------------------	---

3.4 Verification & Quality Layer

3. Submodule Registry (All 58 Submodules)

Submodule	Purpose	Criticality	Upstream	Action Items
LLMsVerifier	Primary verification engine: tests, validation, gatekeeping	P0	vasic-digital/LLMsVerifier	Verify 3-tier model discovery (fix recent). Test all 556 benchmark functions across 40 providers. Validate verifier semaphore and gzip decompression fixes.
HelixQA	Comprehensive QA system with 82 video tests	P0	vasic-digital/HelixQA	Verify all 82 video QA tests pass. Test nil guard fixes. Validate integration with HelixMemory and ensemble output.
Veritas	Truth scoring, fact verification, hallucination detection	P0	vasic-digital/Veritas	Test truth scoring accuracy against labeled dataset. Verify cross-reference validation. Test confidence calibration.
Benchmark	Performance benchmarking and regression detection	P1	vasic-digital/Benchmark	Verify StandardBenchmarkRunner safe.Store migration. Test regression detection sensitivity. Validate baseline framework.
BuildCheck	Build verification and CI gatekeeping	P1	milos85vasic/BuildCheck	Verify P0/P1 regression gate wiring. Test cross-platform build verification. Validate CI validate-all integration.
HelixSpecifier	Specification generation and compliance checking	P0	vasic-digital/HelixSpecifier	Verify spec generation completeness. Test compliance checking against generated specs. Validate .specify directory integration.
Challenges	Challenge/ctf problem generation and validation	P2	vasic-digital/Challenges	Test challenge artifact generation. Verify difficulty calibration. Validate solution checking logic.

3.5 Tooling & Integration Layer

3. Submodule Registry (All 58 Submodules)

Submodule	Purpose	Criticality	Upstream	Action Items
MCP	Model Context Protocol implementation	P0	vasic-digital/MCP	Verify MockMCPTransport safe.Slice migration. Test all 35 MCP server integrations. Validate protocol compliance.
MCP-Servers	Pre-built MCP server collection	P0	vasic-digital/MCP-Servers	Test each MCP server in isolation. Verify interoperability with HelixAgent core. Validate tool schema registration.
MCP_Module	MCP module interface and wiring	P0	vasic-digital/MCP_Module	Verify module initialization paths. Test dynamic tool loading. Validate error handling for unavailable tools.
ToolSchema	JSON Schema validation for tool definitions	P0	vasic-digital/ToolSchema	Test schema validation against all tool definitions. Verify error message clarity. Validate schema evolution handling.
SkillRegistry	Skill discovery, registration, and execution	P0	vasic-digital/SkillRegistry	Verify ProtocolSkillAdapter safe.Store migration. Test skill discovery by capability. Validate skill execution sandbox.
Plugins	Plugin system for extensibility	P1	vasic-digital/Plugins	Verify HealthMonitor safe.Store migration. Test plugin hot-loading. Validate plugin isolation (no panics).

3. Submodule Registry (All 58 Submodules)

Toolkit	Developer tooling and utilities	P1	vasic-digital/Toolkit	Verify all CLI tools function correctly. Test cross-platform compatibility. Validate integration with build system.
Streaming	Server-sent events and streaming response handling	P0	vasic-digital/Streaming	Verify MpscStream atomic.Bool migration. Test SSE under high concurrency. Validate stream cancellation cleanup.
DocProcessor	Document parsing and chunking (PDF, DOCX, etc.)	P1	vasic-digital/DocProcessor	Test parsing for all supported formats. Verify chunking strategy preserves context. Validate metadata extraction.
Formatters	Output formatting and normalization	P1	vasic-digital/Formatters	Verify FormatterRegistry safe.Store migration. Test all output formats (JSON, Markdown, XML). Validate encoding safety.

3.6 Security & Governance Layer

3. Submodule Registry (All 58 Submodules)

Submodule	Purpose	Criticality	Upstream	Action Items
Security	Core security primitives, encryption, signing	P0	vasic-digital/Security	Verify all cryptographic operations. Test secret rotation. Validate secure defaults (no hardcoded keys).
Auth	Authentication and authorization (OAuth2, JWT, API keys)	P0	vasic-digital/Auth	Verify token validation logic. Test RBAC enforcement. Validate session management security.
RedTeam	Adversarial testing and red team exercises	P1	vasic-digital/RedTeam	Verify redteam fixture harness extraction (security fix). Test prompt injection detection. Validate jailbreak resistance.
SelfImprove	Self-improvement and meta-learning loops	P2	vasic-digital/SelfImprove	Verify improvement loop convergence. Test feedback integration. Validate no runaway resource consumption.

3. Submodule Registry (All 58 Submodules)

GandalfSolutions	Advanced security challenge and solution system	P2	vasic-digital/GandalfSolutions	Test generation Verify validation Validate progression.	challenge variety. solution accuracy. difficulty
-------------------------	---	----	--------------------------------	---	--

3.7 Deployment & Operations Layer

3. Submodule Registry (All 58 Submodules)

Submodule	Purpose	Criticality	Upstream	Action Items
Observability	Metrics, logging, tracing, alerting	P0	vasic-digital/Observability	Verify all metrics export correctly. Test trace propagation across submodules. Validate alert thresholds.
Optimization	Performance optimization and resource management	P1	vasic-digital/Optimization	Verify resource limit enforcement (Constitution Rule 15: 30-40% host resources). Test optimizer latency counter fix.
Ouroborous	Self-monitoring and healing system	P1	vasic-digital/Ouroborous	Verify self-healing triggers. Test circuit breaker automation. Validate recovery procedures.
Planning	Task planning and scheduling (MCTS-based)	P1	vasic-digital/Planning	Verify MCTSNode atomic migration. Test planning under resource constraints. Validate plan execution tracking.

3. Submodule Registry (All 58 Submodules)

BackgroundTasks	Background job processing and scheduling	P1	vasic-digital/BackgroundTasks	Verify task retry logic. Test job queue persistence. Validate worker pool scaling.
DebateOrchestrator	Multi-agent debate and consensus building (15-25 LLMs)	P1	vasic-digital/DebateOrchestrator	Test debate convergence. Verify consensus algorithm accuracy. Validate 15-25 LLM coordination.
Agentic	Agent workflow engine and state machine	P0	vasic-digital/Agentic	Verify WorkflowState safe.Store migration. Test agent state transitions. Validate workflow recovery after crash.

3.8 Interface & Experience Layer

3. Submodule Registry (All 58 Submodules)

Submodule	Purpose		Criticality	Upstream	Action Items
Website	React frontend, documentation, user portal	web	P1	vasic-digital/Website	Verify all routes render correctly. Test API integration. Validate provider count sync (drift fix).
VisionEngine	Computer vision image processing	vision image	P2	vasic-digital/VisionEngine	Test image analysis accuracy. Verify model loading performance. Validate multi-format support.
Claritas	Clarity and refinement	scoring output	P2	vasic-digital/Claritas	Test clarity scoring consistency. Verify refinement suggestions. Validate integration with ensemble output.
Normalize	Data normalization and standardization		P2	vasic-digital/Normalize	Verify all normalization pipelines. Test edge case handling. Validate cross-cultural text handling.
LeakHub	Memory detection and analysis	leak and	P2	vasic-digital/LeakHub	Verify leak detection accuracy. Test profiling overhead. Validate integration with test suite.
Upstreams	Upstream dependency tracking and management		P2	(internal)	Verify upstream sync automation. Test dependency vulnerability scanning. Validate update safety.

4. Internal Package Inventory

The `internal/` directory contains **60+ packages** that implement the core business logic. Each package must be individually verified and then integration-tested with its dependencies.

Package Registry by Domain

Domain	Packages	Verification Focus
API & Transport	handlers, http, transport, graphql, router, middleware	Request routing, middleware chain, error handling, rate limiting
LLM Core	llm, llmops, ensemble, models, modelsdev	Provider calls, response parsing, streaming, model discovery
Memory & Context	memory, conversation, knowledge, rag, vectordb, embeddings	Storage, retrieval, caching, context window management
Tools & Skills	tools, skills, plugins, mcp, agentic	Tool execution, skill matching, plugin lifecycle
Security	security, auth, audit, checkpoints	Authentication, authorization, audit logging, checkpoint integrity
Infrastructure	database, cache, storage, messaging, streaming, containers	Connection management, failover, resource limits
Quality & Testing	verifier, testing, testutil, benchmark, challenges	Test accuracy, benchmark validity, challenge generation
Observability	observability, analytics, notifications	Metric collection, alert delivery, dashboard accuracy
Operations	background, services, sanity, version	Health checks, service discovery, boot validation
Utilities	utils, config, formatters, output, templates, search	Config loading, output formatting, template rendering
Platform	browser, clis, adapters, learning, optimization, planning	Browser automation, CLI integration, learning loop convergence
Specialized	bigdata, features, checkpoints, notifications, modelsdev	Big data pipeline, feature flags, checkpoint consistency

CONST-029 MIGRATION VERIFICATION

Many packages have undergone "CONST-029" migration — replacing mutex-protected maps with `safe.Store`, `safe.Slice`, and atomic operations. Every migrated package must be verified for:

- No residual map-based state (grep for `sync.Mutex + map`)
- Thread-safety under race detector (`go test -race`)
- Correct atomic operation ordering
- No deadlocks in concurrent access patterns

5. The Universal Verification Framework

This section implements the eight-step universal approach from the institutional knowledge document, adapted specifically for the HelixAgent monorepo.

5.1 Institutional Knowledge Mapping

Every "landmine" rule must be documented in `docs/LANDMINES.md` with the following structure:

```
## RULE-[DOMAIN]-[NUMBER]: [Short Name]
```

- ****Context:**** Why this rule exists (production incident, architectural constraint)
- ****Detection:**** How to check for violations (file patterns, function names, API endpoints)
- ****Fix:**** The exact remediation required
- ****Test:**** How to add a regression test for this rule
- ****Severity:**** BLOCKER / WARNING / INFO

Example:

```
## RULE-GO-001: Context Cancellation in Handlers
```

- ****Context:**** Using `context.Background()` in HTTP handlers causes goroutine leaks
- ****Detection:**** `grep -r "context.Background()" internal/handlers/`
- ****Fix:**** Always use the request context: `ctx := r.Context()`
- ****Test:**** Add leak detection test with `goleak.VerifyNone()`
- ****Severity:**** BLOCKER

HelixAgent-Specific Landmine Rules (Initial Set)

Rule ID	Name	Detection	Fix	Severity
RULE-GO-001	Context Cancellation	context.Background() in handlers	Use r.Context()	BLOCKER
RULE-GO-002	Database Connection Leaks	Missing defer rows.Close()	Always defer close	BLOCKER
RULE-GO-003	HTTP/2 for TV Endpoints	HTTP/2 protocol for TV routes	Force HTTP/1.1	BLOCKER
RULE-GO-004	Analytics Synchronous Call	analytics.Track() in handler path	Wrap in goroutine	WARNING
RULE-GO-005	Legacy Billing Payload	POST body > 1MB to billing	Chunk into 500 item batches	BLOCKER
RULE-LLM-001	Provider API Key Exposure	API keys in logs or errors	Redact before logging	BLOCKER
RULE-LLM-002	Streaming Cancellation	No cancel on client disconnect	Propagate ctx.Done()	BLOCKER
RULE-MEM-001	Memory Store Sync	Write without read-after-write verify	Confirm persistence	WARNING
RULE-VEC-001	Vector Dimension Mismatch	Embedding dims != index dims	Validate before insert	BLOCKER
RULE-MCP-001	Tool Schema Validation	Tool call without schema check	Validate before execute	BLOCKER
RULE-SEC-001	Hardcoded Secrets	Passwords, keys in source code	Use env/config injection	BLOCKER
RULE-SEC-002	Debug Endpoints in Prod	/debug, /pprof in production	Conditional compilation	BLOCKER
RULE-DEP-001	Resource Limit Violation	Process uses >40% host resources	Enforce cgroups/limits	WARNING
RULE-TEST-001	Mock-Only Verification	Tests use only mocks, no real services	Add integration tests	WARNING

5.2 Testing Pyramid Rebalance

For HelixAgent, the testing pyramid must be restructured from the current unit-test-heavy shape to a balanced, multi-layered defense:

Layer 1: Unit Tests (Current: ~80%, Target: ~40%)

- Keep existing unit tests for algorithmic correctness
- Every test must verify behavior, not implementation
- No mock-only tests for critical paths — use testcontainers for dependencies

Layer 2: Contract Tests (Target: ~20%)

Verify data shape between services:

```
// Example: Provider response contract test
func TestProviderResponseContract(t *testing.T) {
    providers := []string{"openai", "anthropic", "gemini"}
    for _, p := range providers {
        t.Run(p, func(t *testing.T) {
            resp := callProvider(p, "Hello")
            assert.NotEmpty(t, resp.ID)
            assert.NotEmpty(t, resp.Content)
            assert.NotNil(t, resp.Usage)
            assert.Greater(t, resp.Usage.TotalTokens, 0)
        })
    }
}
```

Layer 3: Integration Tests (Target: ~25%)

Test with real dependencies using testcontainers:

```
// Example: Memory + VectorDB integration
func TestMemoryVectorDBIntegration(t *testing.T) {
    pg := testcontainers.RunPostgres(ctx)
    redis := testcontainers.RunRedis(ctx)
    qdrant := testcontainers.RunQdrant(ctx)

    mem := NewHelixMemory(pg, redis, qdrant)
    err := mem.Store(ctx, userID, "test data")
    require.NoError(t, err)

    results, err := mem.Retrieve(ctx, userID, "test")
    require.NoError(t, err)
    require.NotEmpty(t, results)
}
```

Layer 4: End-to-End Tests (Target: ~10%)

Full system tests with docker-compose:

```
# Example: E2E test via Makefile
make test-e2e
# Spins up full stack
# Sends request through API
# Verifies response quality
# Checks memory persistence
# Validates metrics emission
```

Layer 5: Property-Based Tests (Target: ~3%)

Catch edge cases the AI might introduce:

```
// Example: Age calculation never panics
func TestAgeCalculationProperty(t *testing.T) {
    parameters := gopter.DefaultTestParameters()
    parameters.MinSuccessfulTests = 1000
    properties := gopter.NewProperties(parameters)

    properties.Property("never panics",
        prop.ForAll(func(age string) bool {
            _, err := CalculateAge(age)
            return err == nil || err != nil // either is fine, never panic
        }, gen.AnyString()))

    properties.TestingRun(t)
}
```

Layer 6: Chaos Tests (Target: ~2%)

Verify resilience under failure:

- Kill PostgreSQL mid-request — verify graceful degradation
- Drop 50% of provider requests — verify fallback works
- Max out memory — verify OOM handling
- Network partition between services — verify retry and circuit breaker

5.3 Verification Loop Protocol

The self-correcting prompt chain for AI agents working on HelixAgent:

Step 1: Task Assignment Prompt

```
[SYSTEM DIRECTIVE: VERIFICATION MODE ENABLED]
[PROJECT: HelixAgent – HIGH-STAKES ENVIRONMENT]
```

```
TASK: Implement [FEATURE] in [SUBMODULE/PACKAGE].
```

```
REQUIREMENTS:
```

1. Read docs/LANDMINES.md and confirm understanding of applicable rules.
2. Read CLAUDE.md Section 2 for exact build/test commands.
3. Write the code following existing patterns in the same package.
4. Run the FULL verification command for the affected module:
 - For Go: make test-unit && make test-integration && make lint
 - For Docker: make docker-test
 - For Web: cd website && npm run test && npm run build
 - For Android: ./gradlew test lint

```
CONSTRAINT: You are FORBIDDEN from declaring the task complete
until ALL verification commands exit with code 0.
```

```
IF VERIFICATION FAILS:
```

- Step A: Read the complete error output.
- Step B: Fix the root cause (not the symptom).
- Step C: Re-run ALL verification commands.
- Step D: If still failing after 3 attempts, output:
"VERIFICATION BLOCKED: [error summary]"

```
BEGIN TASK.
```

Step 2: Error Feedback Prompt

[CONTINUE VERIFICATION LOOP]

The previous attempt failed:

--- BEGIN ERROR LOG ---

[Paste complete terminal output]

--- END ERROR LOG ---

ANALYSIS REQUIREMENT:

- Explain WHY this error occurred (root cause, not symptom).
- Identify which LANDMINE rule was violated (if any).
- State what you will change to prevent recurrence.

ACTION: Fix the code and re-run the full verification suite.

Step 3: Final Review Checklist

[FINAL REVIEW – BEFORE MERGE]

Verification passed. Generate a Deployment Impact Summary:

1. LANDMINES: Which rules from docs/LANDMINES.md were relevant?
2. API CHANGES: Did you modify any public API contract? (Yes/No + diff)
3. CROSS-CUTTING: Which other submodules/packages are affected?
4. PERFORMANCE: Expected impact on latency, throughput, memory?
5. SECURITY: Any new secrets, network calls, or auth changes?
6. LOW-BANDWIDTH: Expected behavior under degraded network?
7. ROLLBACK: How can this change be reverted if needed?

5.4 LLM-as-Judge Layer

Before any AI-generated code is merged, a second AI instance must review it as a Senior Architect:

Judge Prompt Template

ROLE: Senior Software Architect & QA Gatekeeper

CONTEXT: Reviewing a pull request written by an AI agent for HelixAgent.

INPUT:

- PR Diff: [paste git diff]
- Landmines: [paste docs/LANDMINES.md]
- Architecture: [paste relevant section from CLAUDE.md]

REVIEW CHECKLIST:

1. LANDMINE COMPLIANCE: Any rule violations? (List each with fix)
2. API CONTRACT: Any breaking changes to public interfaces?
3. CONCURRENCY: Any new race conditions or deadlock risks?
4. ERROR HANDLING: Are all errors handled or explicitly ignored?
5. RESOURCE LEAKS: Any missing Close(), Cancel(), or Release()?
6. TEST COVERAGE: Are behavioral tests added, not just unit?
7. CROSS-CUTTING: Any unintended side effects on other modules?
8. PERFORMANCE: Any $O(n^2)$ algorithms or memory explosions?
9. SECURITY: Any injection vectors or secret exposure?

OUTPUT FORMAT (STRICT JSON):

```
{
  "veto": true/false,
  "severity": "BLOCKER" / "WARNING" / "INFO",
  "violations": [
    {
      "rule": "RULE-ID",
      "file": "path/to/file.go",
      "line": 42,
      "description": "...",
      "fix": "..."
    }
  ],
  "risk_assessment": "Low/Medium/High – explanation",
  "recommendations": ["..."]
}
```

CRITICAL: If less than 95% confident this code works in production, you MUST veto: true with severity: BLOCKER.

5.5 CLAUDE.md Operating System

The `CLAUDE.md` file must be treated as the project's operating system manual. It already exists in the repository and must be kept current. Required sections:

Section	Required Content	Update Frequency
System Architecture	High-level diagram, component interactions, critical path	Per architectural change
Build Commands	Exact commands for every output (bin, docker, mobile, web)	Per Makefile change
Test Commands	Unit, integration, e2e, benchmark per module	Per test addition
Institutional Knowledge	Landmine rules with severity and detection	Per incident
Submodule Map	Purpose, upstream, last verified commit of each submodule	Per submodule update
Environment Spec	Go version, Docker version, OS targets, shell	Per tooling change
Security Constraints	Secret handling, auth flows, encryption requirements	Per security review
Challenge Prompt	"Question my assumptions" — counter over-agreeability	Never

5.6 Multi-Agent Review Process

Implement the Three-Lens Model for every significant change:

Lens	Agent Role	Focus	Tools Used
Lens Intent	1: Original author	Does it achieve the feature goal?	Unit tests, manual demo
Lens Strategy	2: Second session AI	Does it fit architecture? Any landmine violations?	CLAUDE.md, LANDMINES.md review
Lens Structure	3: Static analysis + third AI	Code quality, consistency, copy-paste errors	golangci-lint, gosec, go vet, Judge prompt

Rule: The agent that wrote the code is NEVER the same agent that runs Lens 2 or Lens 3. Start a fresh Claude Code session for each review lens.

6. Phase-by-Phase Implementation Plan

The following 11 phases must be executed in order. Each phase has entry criteria, tasks, verification steps, and exit criteria. **No phase may be skipped or declared complete until all exit criteria are met.**

Phase 0: Foundation & Audit

Phase 0 — Foundation & Audit (Week 1)

Goal: Establish the baseline truth of what currently works and what does not.

Entry Criteria: Repository cloned, submodules initialized, Docker available.

Tasks:

1. Run `make test-unit` on entire codebase — record pass/fail per package
2. Run `make test-integration` — record which require live services
3. Run `make lint` — record all lint violations
4. Run `make security-scan` — record gosec, trivy, snyk findings
5. Run `make build-all` — verify cross-compilation for all 5 targets
6. Audit each submodule: is the pointer at a known-good commit?
7. Check all docker-compose files for service interdependencies
8. Verify `.env.example` contains all required variables with defaults
9. Document current test coverage per package (unit vs integration ratio)
10. Create `docs/CURRENT_STATE.md` with the audit results

Exit Criteria:

- Every package has a recorded test result (PASS, FAIL, SKIP, or NO_TESTS)
- Every submodule has a verified commit pointer and upstream URL
- Security scan results are categorized by severity
- Build succeeds for linux-amd64, linux-arm64, darwin-amd64, darwin-arm64, windows-amd64
- Audit document reviewed and signed off by human lead

Phase 1: Verification Infrastructure

Phase 1 — Verification Infrastructure (Week 2)

Goal: Build the mechanical verification system before touching any production code.

Tasks:

1. Create/validate `CLAUDE.md` with all required sections
2. Create `docs/LANDMINES.md` with initial 14 rules (Section 5.1)
3. Create `scripts/ai-dev-loop.sh` automation script
4. Set up testcontainers for PostgreSQL, Redis, Kafka, Qdrant
5. Add contract test framework for provider response shapes
6. Add property-based test utilities (gopter helpers)
7. Add chaos test harness (kill containers mid-test)
8. Create integration test Makefile targets per module
9. Set up LLM-as-Judge automation (GitHub Action or script)
10. Create test result dashboard (HTML or Prometheus metrics)

Exit Criteria:

- `make test-contract` runs and produces JSON report
- `make test-chaos` runs 5 failure scenarios successfully
- Every module has an integration test target in Makefile
- Judge script produces valid JSON output on sample diffs
- Testcontainers spin up in under 30 seconds for standard stack

Phase 2: Core Submodule Hardening

Phase 2 — Core Submodule Hardening (Weeks 3-4)

Goal: Make the foundation rock-solid: concurrency, database, storage, cache, messaging.

Priority Order:

1. **Concurrency** — Verify deadlock detector, WorkerPool, safe.Store primitives
2. **Database** — Verify connection pooling, QueryCache, migrations
3. **Storage** — Verify multi-provider upload/download, multipart resume
4. **Cache** — Verify L1/L2 coherence, stampede protection, tag invalidation
5. **Messaging** — Verify Kafka producer/consumer, DLQ, replay
6. **EventBus** — Verify ordering, subscriber cleanup, backpressure

For Each Submodule:

1. Run full unit test suite with race detector: `go test -race ./...`
2. Run integration tests with real services via testcontainers
3. Run stress test: 10K concurrent operations for 5 minutes
4. Verify CONST-029 migration: no residual map+mutex patterns
5. Run benchmark baseline and check against historical values
6. Perform code review with Lens 2 (Strategy) and Lens 3 (Structure)
7. Document any deviations from expected behavior in `docs/LANDMINES.md`

Exit Criteria:

- All 6 submodules pass unit + integration + stress tests
- Zero data races detected by `go test -race`
- All benchmarks within 10% of baseline or better
- Each submodule has a verified commit pointer documented

Phase 3: Memory & Context Systems

Phase 3 — Memory & Context Systems (Weeks 5-6)

Goal: Ensure memory operations are accurate, persistent, and retrievable under all conditions.

Submodules: HelixMemory, Memory, VectorDB, Embeddings, ConversationContext, RAG

Tasks:

1. HelixMemory:

- Verify fusion duration fix (recent bump)
- Test episodic memory: store -> retrieve -> verify exact match
- Test semantic memory: similar queries return relevant results
- Test distributed mode with Kafka-backed replay
- Verify confidence merging logic

2. VectorDB:

- Test all 4 backends: Qdrant, Weaviate, Chroma, pgvector
- Insert 1M vectors, verify search latency < 100ms
- Test hybrid search (BM25 + vector)
- Verify graceful degradation when vector store is unreachable

3. Embeddings:

- Test all 13 embedding providers with live API calls
- Verify dimension consistency across providers
- Test batch embedding: 10K documents in single call
- Validate normalization preserves semantic similarity

4. RAG:

- End-to-end test: document ingestion -> retrieval -> generation
- Verify AdvancedRAG synonym handling (safe.Store migration)
- Test reranking pipeline accuracy
- Validate chunking strategies (size, overlap, boundary)

5. ConversationContext:

- Test context window overflow handling

- Test user isolation: no cross-user context leakage

Exit Criteria:

- Memory store/retrieve round-trip accuracy > 99%
- Vector search returns relevant results in top-5 for 95% of queries
- All embedding providers respond within 5 seconds
- RAG end-to-end latency < 2 seconds for 1000-document corpus
- No context leakage between users in conversation tests

Phase 4: LLM Provider Integration

Phase 4 — LLM Provider Integration (Weeks 7-8)

Goal: All 47+ providers respond correctly to both streaming and non-streaming requests.

Submodules: HelixLLM, LLMProvider, LLMOrchestrator, LLMOps, I-LLM, Models, HyperTune, AutoTemp

Tasks:

1. Provider Discovery:

- Verify 3-tier model discovery (HelixLLM recent fix)
- Refresh all 10 stale provider configurations
- Test model capability detection for each provider
- Verify provider metadata freshness (auto-refresh)

2. API Compatibility:

- Test each provider with live API call (use test keys)
- Verify streaming response parsing
- Verify non-streaming response parsing
- Test error handling: rate limits, auth failures, timeouts
- Verify fallback chaining when primary provider fails

3. Ensemble Engine:

- Test multi-provider concurrent dispatch
- Verify response aggregation logic
- Test scoring and ranking of responses
- Validate consensus detection (when providers agree)

4. Operational Features:

- Verify circuit breaker behavior under simulated failure
- Test request routing by model capability
- Verify prompt caching (I-LLM/GPTCache)
- Test auto-temperature adjustment (AutoTemp)
- Validate hyperparameter tuning convergence (HyperTune)

5. Load Testing:

- Measure p50, p95, p99 latency
- Verify no goroutine leaks after test
- Check memory usage stays within Constitution limits (30-40%)

Exit Criteria:

- All 47+ providers respond successfully to test prompt
- Streaming and non-streaming paths verified for top 10 providers
- Circuit breaker opens within 5 consecutive failures
- Ensemble produces coherent output for 95% of test queries
- p95 latency < 5 seconds for 100-concurrent load
- Zero goroutine leaks post-test (goleak.VerifyNone)

Phase 5: Tooling & MCP Ecosystem

Phase 5 — Tooling & MCP Ecosystem (Weeks 9-10)

Goal: All tools are discoverable, schema-validated, and executable in isolation.

Submodules: MCP, MCP-Servers, MCP_Module, ToolSchema, SkillRegistry, Plugins, Toolkit, Streaming, DocProcessor, Formatters

Tasks:

1. MCP Protocol:

- Verify MockMCPTransport safe.Slice migration
- Test protocol handshake with all 35 MCP servers
- Verify tool listing, tool call, and tool result exchange
- Test error propagation from tool to caller

2. Tool Schema:

- Validate all tool definitions against JSON Schema
- Test schema evolution (adding optional params)
- Verify error messages for invalid tool calls

3. Skill Registry:

- Verify ProtocolSkillAdapter safe.Store migration
- Test skill discovery by capability keyword
- Verify skill execution sandbox (no panics escape)
- Test skill chaining (output of skill A -> input of skill B)

4. Plugin System:

- Verify HealthMonitor safe.Store migration
- Test plugin hot-loading without restart
- Verify plugin isolation (fault in one doesn't crash others)

5. Streaming:

- Verify MpscStream atomic.Bool migration
- Test SSE under 1000 concurrent connections
- Verify stream cancellation cleans up all resources
- Test backpressure handling

6. Document Processing:

- Test PDF, DOCX, TXT, MD parsing
- Verify chunking preserves semantic boundaries
- Test metadata extraction (title, author, dates)

Exit Criteria:

- All 35 MCP servers respond to protocol handshake
- Tool schema validation catches 100% of invalid invocations
- Skill execution sandbox contains all panics
- Streaming handles 1000 concurrent connections without leak
- Document parsing succeeds for all supported formats

Phase 6: Verification & QA Systems

Phase 6 — Verification & QA Systems (Weeks 11-12)

Goal: The verification systems themselves are verified — meta-quality assurance.

Submodules: LLMsVerifier, HelixQA, Veritas, Benchmark, BuildCheck, HelixSpecifier, Challenges

Tasks:

1. LLMsVerifier:

- Verify generic 3-tier model discovery (recent fix)
- Run all 556 benchmark functions across 40 providers
- Test verifier semaphore fix
- Verify gzip decompression fix
- Validate provider refresh for 10 stale configs

2. HelixQA:

- Run all 82 video QA tests
- Verify nil guard fixes
- Test integration with HelixMemory output
- Validate ensemble output quality scoring

3. Veritas:

- Test truth scoring against labeled fact dataset
- Verify hallucination detection accuracy (>90% precision)
- Test cross-reference validation with web search
- Validate confidence calibration

4. Benchmark:

- Verify StandardBenchmarkRunner safe.Store migration
- Run full benchmark suite
- Verify regression detection sensitivity (catches 5% slowdown)
- Validate baseline framework produces consistent results

5. BuildCheck:

- Verify P0/P1 regression gate wiring
- Test CI validate-all integration

- Validate cross-platform build checks

6. HelixSpecifier:

- Verify spec generation completeness
- Test compliance checking against specs
- Validate .specify directory integration

Exit Criteria:

- 556/556 benchmark functions pass
- 82/82 video QA tests pass
- Veritas hallucination detection > 90% precision on test set
- Benchmark regression detection catches intentional 5% slowdown
- BuildCheck gates prevent merge of code with P0 failures

Phase 7: Security & Governance

Phase 7 — Security & Governance (Weeks 13-14)

Goal: All security controls are functional, audited, and adversarially tested.

Submodules: Security, Auth, RedTeam, SelfImprove, GandalfSolutions

Tasks:

1. Security Scanning:

- Run gosec on all packages — zero HIGH/CRITICAL findings
- Run trivy on all container images — zero CRITICAL CVEs
- Run snyk on dependencies — zero exploitable vulnerabilities
- Run semgrep for custom rules (secret patterns, SQL injection)
- Verify .gosec-baseline.json is current and justified

2. Authentication & Authorization:

- Test JWT validation: expired, malformed, missing signature
- Verify RBAC enforcement for all API endpoints
- Test API key rotation without downtime
- Validate session timeout and cleanup

3. RedTeam:

- Verify redteam fixture harness extraction (security fix)
- Run prompt injection test suite (100+ payloads)
- Test jailbreak resistance across all providers
- Verify adversarial output detection

4. Constitution Compliance:

- Verify CONSTITUTION.json rules are enforced in code
- Test hostname dehardcoding (Const-031 fix)
- Verify .env is the single source of truth for configuration
- Validate resource limit enforcement (30-40% host)

5. Audit Logging:

- Verify all authentication events are logged
- Test audit log tamper resistance
- Verify PII redaction in logs

Exit Criteria:

- Zero HIGH or CRITICAL findings from gosec, trivy, snyk
- All RBAC tests pass — no unauthorized access paths
- RedTeam: < 5% prompt injection success rate
- Audit logs contain all required events with no PII leakage
- Constitution rules are programmatically verifiable

Phase 8: Deployment & Operations

Phase 8 — Deployment & Operations (Weeks 15-16)

Goal: Production deployment is fully automated, observable, and recoverable.

Submodules: Observability, Optimization, Ouroborous, Planning, BackgroundTasks, DebateOrchestrator, Agentic

Tasks:

1. Container Orchestration:

- Test docker-compose.full.yml stack startup (all 24+ services)
- Verify health checks for every container
- Test graceful shutdown sequence
- Verify service discovery (scripts/discover-services.sh)

2. Kubernetes:

- Validate all Helm charts in k8s/ and deployments/
- Test rolling update with zero downtime
- Verify HPA scaling under load
- Test pod disruption budget compliance

3. Observability:

- Verify Prometheus metrics export for all services
- Test Grafana dashboard accuracy
- Verify distributed tracing (Jaeger/Zipkin)
- Validate alert rules (false positive rate < 5%)

4. Self-Healing:

- Verify Ouroborous detects and responds to failures
- Test circuit breaker automation
- Verify automatic restart of unhealthy services

5. Resource Management:

- Verify Optimization enforces 30-40% host resource limit
- Test behavior when limit is exceeded (throttling, not crash)
- Verify cgroup integration on Linux

6. Backup & Recovery:

- Test PostgreSQL point-in-time recovery
- Verify Kafka topic backup/restore
- Test VectorDB index backup

Exit Criteria:

- Full stack starts in < 5 minutes via docker-compose
- Zero-downtime rolling update succeeds
- All metrics are visible in Grafana within 30 seconds of startup
- Ouroborous restores service within 60 seconds of failure
- Resource limits are enforced without crashes
- Database recovery to arbitrary point in time succeeds

Phase 9: Interface & SDK Hardening

Phase 9 — Interface & SDK Hardening (Weeks 17-18)

Goal: All user-facing interfaces work correctly across platforms.

Components: Website, SDK (Android, iOS, Web), VS Code Extension, API

Tasks:

1. Website:

- Verify all routes render without errors
- Test API integration (real requests to backend)
- Validate provider count sync (fix drift)
- Test responsive design on mobile, tablet, desktop
- Verify WebSocket streaming UI updates

2. SDK:

- Test Android SDK integration with sample app
- Verify iOS SDK builds and runs on simulator
- Test Web SDK from browser
- Verify streaming response handling in all SDKs

3. VS Code Extension:

- Test extension activation
- Verify chat panel functionality
- Test code completion integration
- Validate configuration options

4. API Surface:

- Verify all OpenAPI documented endpoints respond correctly
- Test error response format consistency
- Verify rate limiting headers
- Validate CORS configuration

Exit Criteria:

- Website renders on Chrome, Firefox, Safari, Edge without console errors
- SDKs successfully call API and receive responses

- VS Code extension activates and responds to commands
- All API endpoints match OpenAPI specification

Phase 10: Continuous Validation

Phase 10 — Continuous Validation (Ongoing)

Goal: Ensure the system remains correct as code evolves.

Tasks:

1. Set up nightly full test suite run (unit + integration + e2e)
2. Configure benchmark regression alerts (> 5% slowdown triggers investigation)
3. Weekly submodule sync check (all pointers at latest stable)
4. Monthly security scan (gosec, trivy, snyk, semgrep)
5. Quarterly red team exercise (new adversarial tests)
6. Every manual test failure immediately becomes:
 - A new landmine rule in docs/LANDMINES.md
 - A new regression test in the test suite
 - An update to CLAUDE.md if context was missing
7. Track verification metrics:
 - AI code veto rate by judge layer
 - Test escape rate (bugs found in manual testing / total bugs)
 - Mean time to verification pass

Exit Criteria:

- Nightly test suite runs automatically and reports to Slack/email
- Benchmark regression alerts fire within 24 hours of degradation
- Submodule sync automation reduces manual sync to monthly
- Test escape rate < 5% (95% of bugs caught by automated tests)

7. Submodule-Specific Action Items

This section provides **per-submodule checklists** that must be completed for 100% functional verification. Every submodule from Section 3 is expanded here.

LLMsVerifier (Critical — P0)

Recent Fixes to Verify:

- Generic 3-tier model discovery (commit 3cad659)
- Refresh 10 stale provider configurations
- Verifier semaphore fix
- Gzip decompression fix

Test Checklist:

1. Run `go test ./...` in LLMsVerifier submodule — all pass
2. Run benchmark suite: `go test -bench=.` — 556 functions complete
3. Verify model discovery works for T1 (popular), T2 (standard), T3 (niche) models
4. Test provider refresh: simulate stale config, verify auto-refresh
5. Run with race detector: `go test -race` — zero races
6. Verify integration with HelixLLM submodule
7. Test verifier output format matches expected schema
8. Validate error messages are actionable (not generic)

HelixMemory (Critical — P0)

Recent Fixes to Verify:

- Fusion duration and confidence merging (commit d571191)
- Distributed memory with Kafka-backed replay
- Cognee mock server migration to safe.Store

Test Checklist:

1. Store 1000 memories, retrieve by exact match — 100% accuracy
2. Store 1000 memories, retrieve by semantic similarity — top-5 relevance > 90%
3. Test episodic memory timeline reconstruction
4. Test memory expiration (TTL) — expired memories are purged
5. Test distributed mode: write on node A, read on node B
6. Verify Kafka replay reconstructs memory state correctly
7. Test memory compression under high load
8. Verify safe.Store atomic operations (CONST-029 migration)
9. Test memory migration (upgrade path from old format)

HelixSpecifier (Critical — P0)

Test Checklist:

1. Generate spec for each submodule — verify completeness
2. Check compliance of existing code against generated specs
3. Verify .specify directory contains specs for all modules
4. Test spec diff detection (detects when code diverges from spec)
5. Validate spec output format (machine-readable JSON + human Markdown)
6. Verify integration with CI pipeline (spec check gates merge)

HelixLLM (Critical — P0)

Recent Fixes to Verify:

- 3-tier model discovery (generic fix)
- HuggingFace inference URL construction with /models/ path

Test Checklist:

1. Call each of top 10 providers with live API key — verify response
2. Test streaming: receive 100 tokens, verify no drops
3. Test model discovery: list models for each provider, verify non-empty
4. Test fallback: primary provider fails, secondary responds
5. Verify rate limit handling: 429 response triggers backoff
6. Test timeout handling: slow provider is cancelled cleanly
7. Verify response parsing handles all provider formats
8. Test batch requests: multiple prompts in single call

HelixQA (Critical — P0)

Test Checklist:

1. Run all 82 video QA tests — document any failures
2. Verify nil guard fixes prevent panics on edge inputs
3. Test QA scoring against human-labeled quality dataset
4. Verify integration with ensemble output (end-to-end)
5. Test QA under load: 100 concurrent evaluations
6. Validate QA metrics export to Prometheus

Remaining Submodules

Every other submodule follows the same pattern:

1. Run unit tests: `cd [Submodule] && go test ./...`

2. Run with race detector: `go test -race ./...`
3. Verify CONST-029 migration (if applicable): grep for `sync.Mutex` + `map`
4. Run integration tests with real dependencies
5. Verify documentation is complete (README, examples)
6. Check commit pointer is at latest stable upstream
7. Verify no circular dependencies with other submodules

8. Testing Strategy for 100% Functional Correctness

The Five-Layer Defense Model

Layer	Purpose	Tools	Frequency	Owner
1. Static Analysis	Catch syntax, style, security issues before runtime	golangci-lint, gosec, go vet, semgrep	Every commit	CI/CD
2. Unit Tests	Verify individual functions and methods	go test, testify, mockery	Every commit	Developer
3. Contract Tests	Verify data shapes between modules	Custom validators, JSON Schema	Every PR	CI/CD
4. Integration Tests	Verify modules work together with real dependencies	testcontainers, docker-compose	Every PR + Nightly	CI/CD
5. End-to-End Tests	Verify complete user journeys	Selenium, Playwright, custom scripts	Nightly	QA

Critical Test Scenarios (Must All Pass)

Scenario	Setup	Expected Result	Verification
Ensemble query	3+ providers configured	Aggregated response within 5s	Response has content, usage metrics, provider list
Provider outage	Block primary provider IP	Fallback to secondary within 2s	Circuit breaker opens, fallback responds
Memory recall	Store 100 facts, query 1	Correct fact retrieved	Top result matches stored fact
Vector search	Index 10K documents	Relevant docs in top-5	Precision > 90% on test queries
Streaming	SSE request to /chat	Tokens arrive incrementally	All tokens received, connection closes cleanly
Auth failure	Invalid JWT	401 Unauthorized	No data leaked, proper error message
Rate limit	Send 1000 requests/min	429 after limit exceeded	Retry-After header present
Large payload	Send 10MB request	Handled or rejected gracefully	No crash, appropriate error
Database failure	Stop PostgreSQL mid-request	Graceful degradation	Error returned, no goroutine leak
Container restart	Kill core service container	Auto-restart within 60s	Health check passes, requests resume

Test Automation Script

```
#!/bin/bash
# test-full-suite.sh – Run all verification layers

set -euo pipefail

MODULE="${1:-all}"
ATTEMPTS=0
MAX_ATTEMPTS=3

while [ $ATTEMPTS -lt $MAX_ATTEMPTS ]; do
    ATTEMPTS=$((ATTEMPTS + 1))
    echo "=== Verification Attempt $ATTEMPTS ==="

    # Layer 1: Static
    echo "[1/5] Static Analysis..."
    make lint || continue
    make security-scan || continue

    # Layer 2: Unit
    echo "[2/5] Unit Tests..."
    make test-unit || continue

    # Layer 3: Contract
    echo "[3/5] Contract Tests..."
    make test-contract || continue

    # Layer 4: Integration
    echo "[4/5] Integration Tests..."
    make test-integration || continue

    # Layer 5: E2E
    echo "[5/5] E2E Tests..."
    make test-e2e || continue

    echo "=== ALL LAYERS PASSED ==="
    exit 0
done

echo "=== VERIFICATION FAILED after $MAX_ATTEMPTS attempts ==="
exit 1
```

9. Operational Runbooks

Runbook 1: Adding a New Submodule

1. Add to `.gitmodules` with correct upstream URL
2. Run `git submodule update --init`
3. Add to `CLAUDE.md` submodule map with purpose and upstream
4. Create integration test target in Makefile
5. Write contract tests for its public API
6. Add landmine rules if it introduces new constraints
7. Run full verification suite — must pass before merge

Runbook 2: Updating a Submodule Pointer

1. Review upstream changelog for breaking changes
2. Update pointer in parent repo
3. Run submodule-specific tests: `make test-submodule-[NAME]`
4. Run integration tests that depend on this submodule
5. Run full test suite to catch cross-cutting effects
6. Update `CLAUDE.md` with new verified commit
7. Document any new landmine rules from upstream changes

Runbook 3: Deploying to Production

1. Verify all tests pass: `make test-all-must-pass`
2. Run security scan: `make security-scan-all` — zero criticals
3. Build container images: `make docker-build-prod`
4. Scan images: `make security-scan-container`
5. Run benchmark baseline check: `make benchmark-check`
6. Deploy to staging: `make deploy-staging`
7. Run smoke tests against staging

8. Deploy to production with feature flags OFF
9. Gradually enable feature flags (10% -> 50% -> 100%)
10. Monitor metrics for 24 hours before declaring success

Runbook 4: Incident Response — Test Escape

1. Document the bug in `docs/BUGS.md` with reproduction steps
2. Determine which landmine rule would have caught it (or create new rule)
3. Write a regression test that fails before the fix and passes after
4. Fix the bug and run full verification suite
5. Update `docs/LANDMINES.md` with new rule
6. Update `CLAUDE.md` if context was missing
7. Share findings with team to prevent similar escapes

10. Risk Register & Mitigation

Risk ID	Description	Probability	Impact	Mitigation
R001	Submodule drift — pointers out of sync with upstream	High	High	Weekly automated sync check, CI gate for pointer updates
R002	Test takes too long, developers skip integration tests	High	Medium	Parallel test execution, testcontainers caching, nightly full run
R003	Mock-only tests give false confidence	Medium	High	Mandate testcontainers for integration tests, code review check
R004	AI agent ignores CLAUDE.md or LANDMINES.md	Medium	High	Mechanical enforcement via scripts/ai-dev-loop.sh
R005	Performance regression undetected	Medium	Medium	Benchmark baselines with 5% sensitivity, automated alerts
R006	Security vulnerability in dependency	Low	High	Weekly dependency scan (snyk, dependabot), rapid patching process
R007	Cross-platform build failure (Windows, ARM64)	Medium	Medium	CI builds for all targets on every PR, not just Linux AMD64
R008	Provider API changes break integration	High	High	Contract tests run nightly against live APIs, alert on shape changes
R009	Memory/VectorDB corruption under load	Low	High	Chaos tests, stress tests, data integrity verification post-test
R010	Team abandons verification process under pressure	Medium	High	Automated gates in CI, no manual override without escalation

11. Success Metrics & Exit Criteria

Project-Level Exit Criteria

Metric	Current (Estimated)	Target	Measurement
Unit test pass rate	~95%	100%	make test-unit
Integration test pass rate	~60%	100%	make test-integration
E2E test pass rate	~30%	100%	make test-e2e
Test coverage (unit)	~90%	>90%	make test-coverage
Lint violations	Unknown	0	make lint
Security scan (HIGH+)	Unknown	0	make security-scan
Submodule sync status	Partial	100% synced	scripts/check-submodules.sh
Build success (all platforms)	Partial	100%	make build-all
AI code veto rate	N/A	15-25%	Judge layer logs
Test escape rate	High	< 5%	Bugs found manually / total bugs
Mean time to verify	Unknown	< 30 min	CI pipeline duration
Provider API success rate	Unknown	> 95%	Live API test suite
Memory retrieval accuracy	Unknown	> 99%	Round-trip test
Vector search relevance	Unknown	> 90% top-5	Labeled query set
Ensemble coherence	Unknown	> 95%	Human evaluation sample

Definition of "100% Functional"

100% FUNCTIONAL CRITERIA

A feature is 100% functional when ALL of the following are true:

- 1. It compiles without warnings on all target platforms
- 2. All unit tests for the feature pass
- 3. All integration tests involving the feature pass with real dependencies
- 4. The feature works correctly in the E2E test scenario
- 5. Manual testing confirms expected behavior in the target environment
- 6. Performance is within 10% of baseline (or better)
- 7. Security scan shows no HIGH or CRITICAL findings related to the feature
- 8. The LLM-as-Judge approves the implementation (veto: false)
- 9. Documentation is complete and accurate
- 10. A rollback procedure is documented and tested

12. Appendices

Appendix A: Complete Submodule List (Alphabetical)

#	Name	Category	Upstream
1	Agentic	Operations	vasic-digital/Agentic
2	Auth	Security	vasic-digital/Auth
3	AutoTemp	LLM	vasic-digital/AutoTemp
4	BackgroundTasks	Operations	vasic-digital/BackgroundTasks
5	Benchmark	Quality	vasic-digital/Benchmark
6	BuildCheck	Quality	milos85vasic/BuildCheck
7	Cache	Foundation	vasic-digital/Cache

8	Challenges	Quality	vasic-digital/Challenges
9	Claritas	Interface	vasic-digital/Claritas
10	Concurrency	Foundation	vasic-digital/Concurrency
11	Containers	Foundation	vasic-digital/Containers
12	ConversationContext	Memory	vasic-digital/conversation
13	Database	Foundation	vasic-digital/Database
14	DebateOrchestrator	Operations	vasic-digital/DebateOrchestrator
15	DocProcessor	Tooling	vasic-digital/DocProcessor
16	Embeddings	Memory	vasic-digital/Embeddings
17	EventBus	Foundation	vasic-digital/EventBus
18	Formatters	Tooling	vasic-digital/Formatters
19	GandalfSolutions	Security	vasic-digital/GandalfSolutions
20	HelixLLM	LLM	vasic-digital/HelixLLM
21	HelixMemory	Memory	vasic-digital/HelixMemory
22	HelixQA	Quality	vasic-digital/HelixQA
23	HelixSpecifier	Quality	vasic-digital/HelixSpecifier
24	HyperTune	LLM	vasic-digital/HyperTune
25	I-LLM	LLM	vasic-digital/I-LLM
26	LLMOps	LLM	vasic-digital/LLMOps
27	LLMOrchestrator	LLM	vasic-digital/LLMOrchestrator
28	LLMProvider	LLM	vasic-digital/LLMProvider
29	LLMsVerifier	Quality	vasic-digital/LLMsVerifier
30	LeakHub	Interface	vasic-digital/LeakHub
31	MCP	Tooling	vasic-digital/MCP
32	MCP-Servers	Tooling	vasic-digital/MCP-Servers
33	MCP_Module	Tooling	vasic-digital/MCP_Module
34	Memory	Memory	vasic-digital/Memory
35	Messaging	Foundation	vasic-digital/Messaging

36	Models	LLM	vasic-digital/Models
37	Normalize	Interface	vasic-digital/Normalize
38	Observability	Operations	vasic-digital/Observability
39	Optimization	Operations	vasic-digital/Optimization
40	Ouroborous	Operations	vasic-digital/Ouroborous
41	Planning	Operations	vasic-digital/Planning
42	PliniusCommon	Foundation	vasic-digital/PliniusCommon
43	Plugins	Tooling	vasic-digital/Plugins
44	RAG	Memory	vasic-digital/RAG
45	RedTeam	Security	vasic-digital/RedTeam
46	Security	Security	vasic-digital/Security
47	SelfImprove	Security	vasic-digital/SelfImprove
48	SkillRegistry	Tooling	vasic-digital/SkillRegistry
49	Storage	Foundation	vasic-digital/Storage
50	Streaming	Tooling	vasic-digital/Streaming
51	ToolSchema	Tooling	vasic-digital/ToolSchema
52	Toolkit	Tooling	vasic-digital/Toolkit
53	Upstreams	Interface	(internal)
54	VectorDB	Memory	vasic-digital/VectorDB
55	Veritas	Quality	vasic-digital/Veritas
56	VisionEngine	Interface	vasic-digital/VisionEngine
57	Website	Interface	vasic-digital/Website
58	admin	Interface	(internal)

Appendix B: Internal Package List (Alphabetical)

#	Package	Domain
1	internal/adapters	Platform

12. Appendices

2	internal/agentic	Tools
3	internal/agents	Tools
4	internal/analytics	Observability
5	internal/audit	Security
6	internal/auth	Security
7	internal/background	Operations
8	internal/benchmark	Quality
9	internal/bigdata	Specialized
10	internal/browser	Platform
11	internal/cache	Infrastructure
12	internal/challenges	Quality
13	internal/checkpoints	Security
14	internal/clis	Platform
15	internal/concurrency	Foundation
16	internal/config	Utilities
17	internal/containers	Infrastructure
18	internal/conversation	Memory
19	internal/database	Infrastructure
20	internal/embeddings	Memory
21	internal/ensemble	LLM Core
22	internal/features	Specialized
23	internal/formatters	Utilities
24	internal/graphql	API
25	internal/handlers	API
26	internal/http	API
27	internal/knowledge	Memory
28	internal/learning	Platform
29	internal/llm	LLM Core

30	internal/llmops	LLM Core
31	internal/mcp	Tools
32	internal/memory	Memory
33	internal/messaging	Infrastructure
34	internal/middleware	API
35	internal/models	LLM Core
36	internal/modelsdev	Specialized
37	internal/notifications	Observability
38	internal/observability	Observability
39	internal/optimization	Platform
40	internal/output	Utilities
41	internal/planning	Operations
42	internal/plugins	Tools
43	internal/rag	Memory
44	internal/router	API
45	internal/sanity	Operations
46	internal/search	Utilities
47	internal/security	Security
48	internal/services	Operations
49	internal/skills	Tools
50	internal/storage	Infrastructure
51	internal/streaming	Infrastructure
52	internal/templates	Utilities
53	internal/testing	Quality
54	internal/testutil	Quality
55	internal/tools	Tools
56	internal/transport	API
57	internal/utils	Utilities

58	internal/vectordb	Memory
59	internal/verifier	Quality
60	internal/version	Operations

Appendix C: CONST-029 Migration Verification Checklist

CONST-029 is the migration from mutex-protected maps to `safe.Store/safe.Slice` and atomic operations. Verification must confirm:

1. Search for residual patterns:

```
grep -r "sync.Mutex" --include="*.go" [package] | grep -i "map"
```

Result must be empty for migrated packages.

2. Race detector pass:

```
cd [package] && go test -race ./...
```

Must pass with zero races.

3. Atomic operation audit:

```
grep -r "atomic\." --include="*.go" [package]
```

Every atomic operation must have a comment explaining why it is correct.

4. Load test: Run 10K concurrent operations for 5 minutes. Must complete without deadlock or panic.

Appendix D: Makefile Target Reference

Target	Purpose	When to Run
<code>make all</code>	fmt + vet + lint + test + build	Pre-commit sanity check
<code>make build</code>	Build helixagent binary	Local development
<code>make build-all</code>	Cross-compile all platforms	Release preparation
<code>make test</code>	Run all tests	Pre-PR verification
<code>make test-unit</code>	Unit tests only	Fast feedback loop
<code>make test-integration</code>	Integration tests	Pre-PR, requires Docker
<code>make test-e2e</code>	End-to-end tests	Nightly, requires full stack
<code>make test-coverage</code>	Coverage report	Weekly audit
<code>make test-bench</code>	Benchmarks	Performance validation
<code>make lint</code>	golangci-lint	Every commit
<code>make fmt</code>	gofmt all code	Pre-commit
<code>make vet</code>	go vet	Pre-commit
<code>make security-scan</code>	gosec + trivy + snyk	Pre-PR, nightly
<code>make docker-full</code>	Full docker-compose stack	Local integration testing
<code>make docker-build-prod</code>	Production container image	Release
<code>make release</code>	Build release for all platforms	Release
<code>make benchmark-baseline</code>	Record performance baseline	After performance optimization
<code>make benchmark-check</code>	Check against baseline	Nightly
<code>make audit</code>	Dead code + coverage audit	Monthly cleanup

Appendix E: Verification Command Cheat Sheet

```
# Full verification suite (run before ANY merge)
make fmt && make vet && make lint && make test-unit && \
    make test-integration && make test-e2e && make security-scan

# Quick check (for rapid iteration)
make fmt && make vet && make test-unit

# Per-package verification (example: verifier)
cd internal/verifier && go test -race ./... -count=1

# Submodule verification (example: LLMsVerifier)
cd LLMsVerifier && go test ./... && go test -bench=. -run=^$

# Docker stack verification
make docker-full && sleep 30 && make test-e2e

# Security scan (all layers)
make security-scan-gosec && make security-scan-trivy && \
    make security-scan-snyk && make security-scan-semgrep

# Race detector (critical for CONST-029 packages)
go test -race ./... -count=1 -timeout=30m

# Memory leak detection (requires goleak)
go test ./... -count=1 && goleak.VerifyNone(t)
```

Appendix F: LLM-as-Judge Automation Script

```
#!/bin/bash
# judge.sh – Automated LLM-as-Judge gatekeeper

DIFF_FILE="/tmp/pr_diff.txt"
JUDGE_OUTPUT="/tmp/judge_result.json"

# Export diff
git diff main...HEAD > "$DIFF_FILE"

# Call judge (replace with actual Claude Code or API call)
claude --prompt "
ROLE: Senior Software Architect
INPUT:
- Diff: $(cat "$DIFF_FILE")
- Landmines: $(cat docs/LANDMINES.md)
TASK: Review diff for landmine violations, API contract changes,
      concurrency risks, error handling, resource leaks,
      test coverage, performance issues, security flaws.
OUTPUT: Strict JSON with veto, severity, violations[].
" > "$JUDGE_OUTPUT"

# Parse result
VETO=$(jq -r '.veto' "$JUDGE_OUTPUT")
if [ "$VETO" = "true" ]; then
    echo "JUDGE VETOED. Merge blocked."
    jq '.' "$JUDGE_OUTPUT"
    exit 1
fi

echo "JUDGE APPROVED. Proceed with merge."
exit 0
```

Appendix G: Team Communication Protocol

Daily Standup (15 min):

- What module did you verify yesterday?
- What module will you verify today?
- Any blockers or test escapes discovered?

Weekly Review (60 min):

- Review test escape rate and new landmine rules added
- Review benchmark trends (any regression?)
- Update CLAUDE.md if context gaps were found
- Plan submodule sync strategy for next week

Phase Gate Review (per Phase from Section 6):

- Human lead reviews all exit criteria
- Run full verification suite live in meeting
- Sign off on `docs/PHASE_[N]_COMPLETE.md`
- No phase proceeds without sign-off

FINAL REMINDER

This plan is designed to be **mechanical and repeatable**. The AI agents must not "improvise" or "optimize" their way around these steps. Every task assignment must include the verification loop constraint. Every merge must pass the LLM-as-Judge gate. Every bug found in manual testing must close the feedback loop by updating the institutional knowledge.

Code that compiles is not code that works. Trust only what is verified.